

MLOps Course — End-Term Project

Stock Market Prediction

An End-to-End MLOps Pipeline for NIFTY 100 Next-Minute Price Prediction

Technical Report

Team 44

Roll No	Name	GitHub	Email
DA25G504	Anish Tuteja	AnishTuteja	anishtuteja28@gmail.com
DA25M542	Ambeth Krishnan	aakk27	k.ambeth@gmail.com
DA25M565	Gargi Gupta	gargigazy	gargi.gupta@keysight.com
DA25M622	Sohang Chopra	sohang3112	sohangchopra@gmail.com

Item	Value
Project title	Stock Market Prediction
Repository	https://github.com/sohang3112/stock-prediction-mlops
Branch	main
Dataset	NIFTY 100 index, 1-minute OHLCV bars, Jan 2015 – May 2026 (Git LFS)
Report date	22 August 2026

1. Problem Statement

Intraday index prices move every minute, and the next move is notoriously hard to forecast. The goal of this project is not to build a trading strategy but to demonstrate a production-style machine-learning system around a realistic, noisy time-series problem: given the most recent one-minute OHLC bars of the NIFTY 100 index, predict the index close one minute ahead.

Concretely, the model learns to predict the change in close price between the current bar and the next bar (a differencing / return target); the absolute forecast is reconstructed by adding the predicted change to the current close. This framing keeps the target stationary and prevents the model from simply memorising price levels, a mistake that produced a negative R^2 in our very first baseline.

Why this problem suits an MLOps exercise. The dataset is large enough (about 1.05 million rows) to justify a distributed engine, it arrives as a static historical file (which makes the pipeline fully reproducible), and the prediction task is simple enough that the emphasis can stay on the operational pieces the course rubric asks for: version control, automated data pipeline, experiment tracking, containerised deployment, monitoring and CI/CD.

1.1 Scope and deliverables

- **Automated pipeline:** an Airflow DAG orchestrating ingest → validate → preprocess → train → evaluate → register, with PySpark as the compute engine.
- **Two compared models:** Spark MLlib RandomForestRegressor and LinearRegression (plus a persistence baseline in the Airflow track), tracked in MLflow.
- **Serving:** a FastAPI prediction service packaged in Docker and brought up with docker compose after the training jobs complete.
- **Monitoring:** per-request MLflow prediction/latency logging, prediction request & response logged to MLflow, and an in-process performance-monitoring endpoint.
- **CI/CD:** GitHub Actions running pre-commit (Ruff, mypy, pytest) and a full docker compose integration run on every push and pull request.

2. Dataset Description

The dataset is the historical minute-level price series of the **NIFTY 100** index, stored as a single CSV file `data/raw/NIFTY 100_minute.csv` (~57 MB, 1,048,291 data rows). It spans **9 January 2015 to 15 May 2026**, one row per trading minute, with the following columns:

Column	Type (raw)	Meaning	Notes
date	string	Bar timestamp, yyyy-MM-dd HH:mm:ss (naive IST)	Parsed to timestamp; ~10.5k rows carry non-zero seconds
open	string → double	Open price of the minute	
high	string → double	High price of the minute	
low	string → double	Low price of the minute	
close	string → double	Close price of the minute	Used to build the target
volume	string → double	Traded volume	Always 0 (it is an index) — dropped as a feature in the pipeline

Data characteristics. Prices range from roughly 8,300 in 2015 to about 24,700 in 2026, so absolute price levels are strongly non-stationary while minute-to-minute changes are small (typical $|\Delta \text{close}|$ of a few points). Bars exist only inside trading sessions (09:15–15:30 IST), so naive rolling windows would otherwise leak across the overnight gap — the pipeline partitions all window functions by trading session. There is no class imbalance to handle

because this is a regression problem; the relevant data-quality issues are null timestamps/closes, incomplete windows at the start of each session, and the constant-zero volume column.

Storage and versioning. The CSV is tracked with **Git LFS** so the repository stays light while the exact dataset version is pinned to every commit (see Section 6 for the LFS-vs-DVC rationale). A fresh clone only needs `git lfs install` & `git lfs pull` to obtain it.

3. System Architecture

The system is organised in five layers: orchestration (Airflow), data and artifact storage (parquet on disk, Git LFS, MLflow tracking + Model Registry), compute (PySpark / Spark MLlib), serving (FastAPI in Docker) and cross-cutting concerns (monitoring, CI/CD, version control).

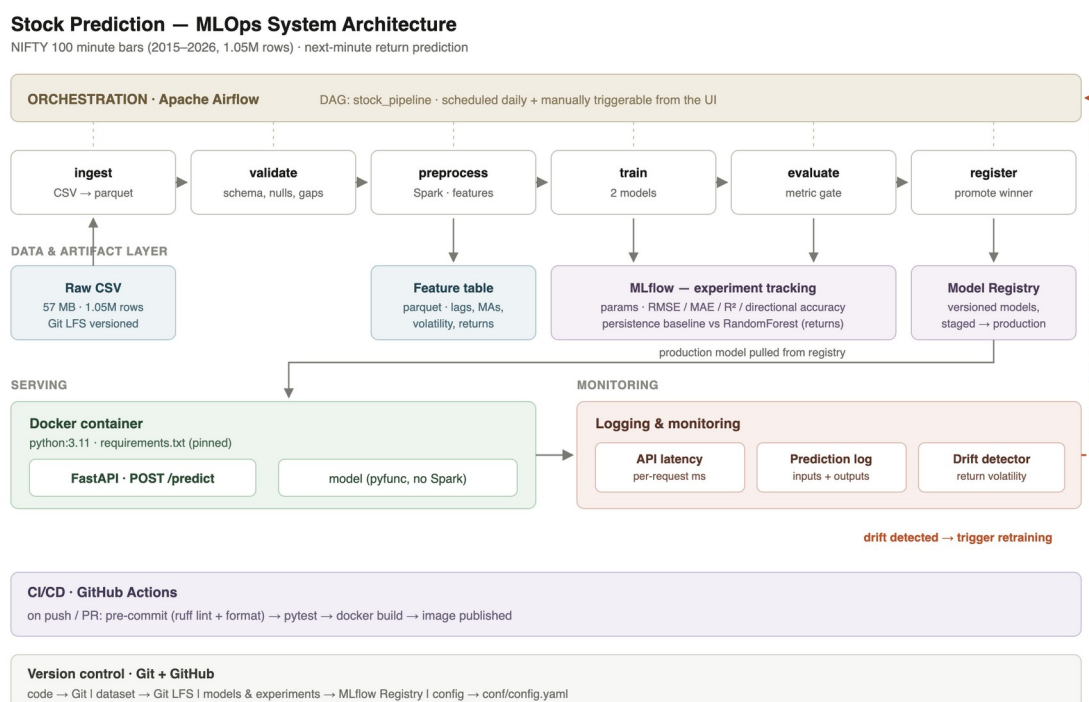


Figure 1 — System architecture

3.1 Components and responsibilities

Layer	Technology	Role in this project
Orchestration	Apache Airflow 3.3 (BashOperator)	DAG stock_pipeline runs the six stages sequentially; a failing validate stops everything downstream.
Data engineering	PySpark 4.2 / Spark MLlib	CSV → parquet ingest, session-partitioned window features, model training on ~840k rows.
Experiment tracking	MLflow 3.15 (SQLite backend)	Params, metrics (RMSE/MAE/ R^2 /directional accuracy), Spark model artifacts, Model Registry with a serving alias, per-request prediction logging.
Data versioning	Git LFS	Pins the 57 MB raw CSV to each commit; models are versioned in the MLflow registry instead.
Serving	FastAPI 0.136 + Uvicorn	POST /predict returns the next-minute close and latency; OpenAPI docs at /docs.
Containerisation	Docker (python:3.14 + OpenJDK 21), docker compose	Same image trains both models and serves; compose chains train → train → serve.
Monitoring	MLflow runs, structured logs, /metrics endpoint	Latency percentiles, prediction distribution, error rate and uptime.

Layer	Technology	Role in this project
CI/CD	GitHub Actions + pre-commit	Ruff lint/format, mypy, pytest, and a full docker compose up on every push/PR.
Configuration	conf/config.yaml	Single source of truth for paths, split, features, hyper-parameters, MLflow and Spark settings.

3.2 Why Spark (and not Kafka or Beam)

The course requires Airflow plus at least one of Spark, Kafka or Beam. We chose Apache Spark because the problem is a batch, historical one: a single immutable file of roughly one million rows that must be parsed, windowed (lags, moving averages, next-bar label) and used to fit models. Spark's DataFrame window functions express the session-partitioned feature engineering in a few lines and run the same code on a laptop in local mode or on a cluster, and Spark MLlib provides the regressors, VectorAssembler/StandardScaler pipeline and PipelineModel serialisation that the serving container loads. Kafka would only add value if live minute bars were being streamed in, and Beam would add an abstraction layer without a runner we needed. Streaming ingestion with Kafka is listed as a future improvement.

3.3 Key architectural decisions

Design decisions are recorded as ADR identifiers in code comments and the configuration file. The most important ones:

- **Chronological train/test split, never random** (ADR-001): a random split on a time series leaks future bars into training and inflates every metric.
- **Differencing target** (ADR-002): the model predicts the change/difference (`next_close - close`), not the price itself.
- **SQLite-backed MLflow store** (ADR-006): the Model Registry is unavailable on a plain file store; serving resolves models: `/<name>@<alias>`.
- **Windows partitioned by trading session** (ADR-012) and **volume dropped** (ADR-013) because it is identically zero for an index.
- **Spark session timezone pinned to Asia/Kolkata** (ADR-205) so the same CSV yields byte-identical parquet on an IST laptop and a UTC container.

4. Data Pipeline

4.1 Airflow DAG

The DAG `stock_pipeline` (`dags/stock_pipeline_dag.py`) defines six strictly sequential tasks. Each task is a BashOperator that runs one Python module from the repository root with `--config conf/config.yaml`, so the orchestrator and the Spark environment version independently. The DAG is `schedule=None` (manual trigger, ADR-200) with `catchup=False` and `max_active_runs=1`. Because the default trigger rule is `all_success`, a contract violation in `validate` halts `train/evaluate/register` with no extra code.

#	Task	Module	Input → Output	What it does
1	ingest	<code>src.data.ingest</code>	raw CSV → <code>data/interim/bars.parquet</code>	Parse timestamps, derive <code>session_date</code> , cast OHLC to double, drop null ts/close, sort by time.
2	validate	<code>src.validate --stage bars</code>	<code>bars.parquet</code> → <code>reports/quality_bars.json</code>	Assert the BARS data contract via PyArrow metadata; write row count, time coverage, session count.
3	preprocess	<code>src.data.preprocess</code>	<code>bars.parquet</code> → <code>data/processed/features.parquet</code>	Session-partitioned lags, diffs, 5/10-bar MA deviations, returns, next-bar label; drop incomplete windows.

#	Task	Module	Input → Output	What it does
4	train	src.models.train	features.parquet → MLflow runs + predictions.parquet	Time split, fit RandomForest pipeline, log persistence + RF runs, write per-row test predictions.
5	evaluate	src.models.evaluate	predictions.parquet → reports/metrics.json	Score RMSE/MAE/R ² (target and price space) and directional accuracy; gate on ≥50% directional accuracy.
6	register	src.models.register	metrics.json → MLflow Model Registry	Register the winning run as nifty-return-predictor and set the production alias.

Data contract. `src/schema.py` declares the RAW, BARS, FEATURES and PREDICTIONS schemas as frozen dataclasses and is imported by every stage, including serving. Each stage validates its input and output against the contract (Spark DataFrames in-process, parquet files via PyArrow metadata), and the validate task fails the DAG listing every violation at once rather than stopping at the first.

4.2 Data processing

Requirement	Implementation
Data cleaning	Explicit string→double casts (no inferSchema), timestamp parsing with a fixed pattern, rows with null timestamp or close dropped at ingest and counted in the log.
Missing values	Not imputed (ADR-106/207): lag and window features are undefined for the first bars of each session, so those rows are dropped (<code>bar_in_session < 10</code>) and any remaining null feature/label rows are removed with <code>dropna</code> ; the counts are printed per stage.
Feature engineering	<code>diff_open/high/low/close</code> (current – previous bar), <code>diff_ma_5</code> and <code>diff_ma_10</code> (close – rolling mean within the session), <code>ret_prev</code> (simple return), <code>diff_volume</code> in the compose training script. Target = <code>next_close – close</code> ; <code>label_original</code> keeps the absolute next close for price-space evaluation.
Preprocessing	<code>VectorAssembler</code> → <code>StandardScaler</code> (withStd, no mean-centering to keep sparse-friendly) inside the Spark ML Pipeline, so the same transformation is serialised with the model and applied at serving time.
Class imbalance / augmentation	Not applicable — regression on a continuous target.
Train/test split	Chronological 80/20 cut on the epoch-seconds range (838,854 train / 209,435 test rows); the cutoff is computed in epoch seconds so it does not move between IST and UTC hosts.

Configuration-driven. All paths, the split fraction, lag columns, moving-average windows, hyper-parameters, the evaluation gate, MLflow URIs and Spark settings live in `conf/config.yaml`. Changing a hyper-parameter or the gate threshold during the viva means editing this file, not code.

4.3 Running the pipeline

Each stage is also a CLI, so the whole pipeline can be run without Airflow (this is exactly what the DAG executes):

```
python -m src.data.ingest --config conf/config.yaml
python -m src.validate --config conf/config.yaml --stage bars
python -m src.data.preprocess --config conf/config.yaml
python -m src.models.train --config conf/config.yaml
python -m src.models.evaluate --config conf/config.yaml
python -m src.models.register --config conf/config.yaml
```

5. Model Development

Two learned models are trained and compared on identical features and the identical chronological split by `models/train_pyspark_models.py --model {random_forest|linear_regression}`, which is what docker compose runs. The Airflow track additionally scores a **persistence baseline** (predicted change = 0, i.e. next close = current close), which is the reference any model must beat on a near-random-walk series.

Model	Library / estimator	Hyper-parameters	Rationale
Random Forest	Spark MLlib RandomForestRegressor	50 trees, maxDepth 10, featureSubsetStrategy sqrt, subsamplingRate 0.7	Non-linear, robust to feature scale and outliers, captures interactions between OHLC differences.
Linear Regression	Spark MLlib LinearRegression	maxIter 100, regParam 0, elasticNetParam 0	Fast, interpretable linear baseline on stationary (differenced) features.
Persistence (baseline)	none (rule)	—	Random-walk reference: abstains from a directional call; sets the bar for RMSE.

Feature pipeline. Every model is a Spark ML Pipeline of VectorAssembler → StandardScaler → estimator, so scaling parameters are fitted on the training split only and persisted with the model (data/output/<model>/ as a PipelineModel and as an MLflow Spark model artifact).

5.1 Evaluation metrics

- **RMSE and MAE** on the reconstructed next-minute close (price space) — the numbers reported by the compose training jobs — and on the predicted change (target space) in the evaluate stage.
- **R²** in both spaces. Price-space R² is ≈0.9999 for every model because close dominates close + Δ; target-space R² and directional accuracy are what separate models (ADR-004/208).
- **Directional accuracy** — share of correct sign calls among rows where the model commits to a direction; the promotion gate requires at least 50%, i.e. better than a coin flip.
- **pred_std % of actual** — detects a model collapsing to the mean.

5.2 Experiment tracking with MLflow

All training runs log to the experiment `nifty-stock-prediction`. For each model run we log the parameters (model type, hyper-parameters, train fraction, feature list), the metrics (train/test row counts, RMSE, MAE, R²), and the fitted PipelineModel via `mlflow.spark.log_model`. Serving resolves `models:/nifty-return-predictor@production` so a new model version can be promoted without rebuilding the image.

At inference time the FastAPI service opens a short MLflow run `prediction_request` per call and logs `prediction_latency_ms`, `predicted_close`, `current/previous close` and `model_name` — giving a queryable audit trail of what the model predicted and how fast.

6. Dataset and Model Versioning: Git LFS instead of DVC

The guidelines ask for "DVC or an equivalent tool" for dataset/model versioning. We deliberately chose Git LFS for the dataset. The reasoning:

1. **The dataset is a single, immutable historical file.** There is one 57 MB CSV that never changes during the project. DVC shines when data is regenerated often, lives in many files, or must be shared through object storage; none of that applies here. LFS gives exactly the property we need — every commit pins a specific content hash of the file — with no extra moving parts.
2. **No separate remote to provision or secure.** DVC stores pointers in Git but the bytes in a remote (S3, GCS, Google Drive, SSH...) that each teammate must configure and authenticate against. Git LFS uses GitHub's

built-in LFS storage, so a clone plus `git lfs pull` is the entire onboarding — important for a four-person team and for examiners reproducing the project.

3. **One tool, one workflow.** LFS is transparent: the file is checked out, diffed, branched and merged with ordinary git commands, and CI (GitHub Actions checkout) fetches it automatically. DVC adds a second CLI, `.dvc` pointer files, `dvc pull/push` steps and a cache to explain and maintain.
4. **Pipeline and metrics tracking are already covered.** DVC's other selling points — `dvc repro` DAGs and `dvc metrics` — are provided here by Airflow and MLflow respectively, so adopting DVC would duplicate functionality rather than add it.

Trade-offs we accept. GitHub LFS has a storage/bandwidth quota (1 GB free), it cannot deduplicate or diff large binaries, and it is not designed for multi-gigabyte, frequently changing datasets. If the project grew to streaming or daily-refreshed data, DVC (or lakeFS) with an object-store remote would be the right upgrade; this is listed under future improvements.

7. Deployment Strategy

7.1 Container image

A single Dockerfile builds one image used for both training and serving: `python:3.14` base, OpenJDK 21 installed for Spark, pinned `requirements.txt` (`pyspark 4.2.0`, `mlflow 3.15.1`, `fastapi[standard] 0.136.3`, ...), the repository copied to `/app`, default command `fastapi run prediction_server.py`. Using one image removes any drift between the environment that trains a model and the one that loads it.

7.2 docker compose topology

`docker-compose.yml` declares three services on a shared network with the `data/` and `mlflow_data/` directories bind-mounted:

Service	Command	Depends on	Purpose
train-random-forest	<code>python models/train_pyspark_models.py --model random_forest</code>	—	Trains + evaluates RF, logs to MLflow, saves PipelineModel to <code>data/output/random_forest</code>
train-linear-regression	<code>python models/train_pyspark_models.py --model linear_regression</code>	train-random-forest (service_completed_successfully)	Same for Linear Regression
serve	<code>fastapi run prediction_server.py (port 8000)</code>	train-linear-regression (service_completed_successfully)	Loads both PipelineModels at start-up and serves <code>/predict</code>

The `service_completed_successfully` conditions turn compose into a tiny pipeline: the API only starts once both models exist, and a training failure stops the stack with a non-zero exit — which is also what the CI workflow relies on. Spark memory is capped at 1 GB driver/executor via environment variables so the stack runs on a laptop and on the GitHub Actions runner.

```
docker compose up --build    # train RF -> train LR -> serve on :8000
docker compose logs -f serve
docker compose down
```

7.3 API

`POST /predict?model_name=random_forest|linear_regression` takes the current and previous one-minute bars as JSON (validated by a pydantic StockOHLCV model) and returns the predicted next close, the model used and the server-side latency. Interactive documentation is served at `/docs`.


```
curl -X POST 'http://localhost:8000/predict?model_name=random_forest' \
-H 'Content-Type: application/json' -d '{
  "current": {"open": 8200.5, "high": 8300, "low": 8200.2, "close": 8200.8, "volume": 0},
  "previous": {"open": 8100.3, "high": 8200.5, "low": 8100, "close": 8200.5, "volume": 0}}'
```

Response JSON: {"predicted_next_close": 8200.2005, "model_name": "random_forest", "latency_ms": 987.12}

8. Monitoring Strategy

Monitoring covers three layers, satisfying the rubric's "API latency / prediction logging" and "performance monitoring" options:

Layer	Mechanism	What is captured
Prediction & latency logging	MLflow run per /predict call (experiment nifty-stock-prediction, run name prediction_request)	prediction_latency_ms, predicted_close, current_close, previous_close, model_name — queryable in the MLflow UI or via the client API.
Structured application logs	Python logging (INFO) from the FastAPI process, captured by docker compose logs	Request parameters and predictions as JSON-like dicts, Spark/MLflow warnings, model-load events.

9. CI/CD Implementation

Following GitHub Actions workflow run on every push and pull request; both badges are shown at the top of the README.

Workflow	Trigger	Steps	What it guarantees
pre-commit (.github/workflows/pre-commit-action.yaml)	pull_request, push to main	checkout → setup-python 3.13 (pip cache) → pip install -r requirements.txt → pre-commit/action: ruff-check --fix, ruff-format, mypy (with types-PyYAML, pydantic)	Code style, static typing

Local gate. The same hooks are installed locally with pre-commit install, so most failures are caught before a push. Continuous delivery is intentionally simple: merging to main produces a green compose run that proves the current image is deployable; pushing the image to a registry is listed as future work.

9.1 Version-control practice

The repository (<https://github.com/sohang3112/stock-prediction-mlops>, branch main) has 59 commits from all four members over 26 July – 21 August 2026, organised as feature branches merged through 7 pull requests (for example #2 MLflow tracking + prediction endpoint, #3 differencing baseline, #4 track-b-airflow, #6 track-c-serving, #7 model_fixes, #8 feature/PerformanceMonitoring). Commit messages describe the change and, where relevant, the bug fixed; design decisions are captured as ADR references in code.

10. Results and Discussion

The numbers below are from the docker compose run on the main branch (1,048,289 feature rows after dropping lag/label nulls; chronological split 838,854 train / 209,435 test rows, the test period being roughly the last 2¼ years of the series). Metrics are computed on the reconstructed next-minute close.

Model	Test RMSE (points)	Test MAE (points)	Test R ²	Serving latency (single request)
Random Forest (50 trees, depth 10)	10.803	5.634	0.999925	≈ 990 ms (first call, includes Spark job warm-up)
Linear Regression	10.782	5.616	0.999925	≈ 150 ms

Discussion.

- **Both models are essentially tied**, and the linear model is marginally better on RMSE/MAE while being roughly six times faster to serve. On a one-minute horizon the differenced OHLC features carry little exploitable signal, so the extra capacity of the forest buys nothing; in production we would prefer the linear model for cost and simplicity, or keep both behind the `model_name` switch for A/B comparison as we do now.
- **Price-space R^2 is not a useful discriminator**. An R^2 of 0.99992 looks spectacular but is almost entirely explained by `close ≈ next_close`; that is why the Airflow evaluate stage also reports target-space R^2 , directional accuracy and the persistence comparison (ADR-004/208), and why promotion is gated on directional accuracy rather than R^2 .
- **Differencing was decisive**. Our first baseline predicted the absolute price from absolute OHLC levels and scored $RMSE ≈ 3,098$ and $R^2 ≈ -5.2$ on the test period, because 2024–2026 price levels were outside the training range. Switching to stationary features and a price-change target (PR #3) brought RMSE down to $≈ 10.8$ points.
- **MAE of $≈ 5.6$ points on an index near 24,000** is a relative error of about 0.02%, consistent with the typical one-minute move; i.e. the models are about as good as persistence, which is the honest expected outcome for minute-level index forecasting without order-book or news features.
- **Operational results**. The full stack (build, two trainings, API start) completes in a few minutes on a laptop and in GitHub Actions; every run is reproducible from a clean clone thanks to pinned requirements, the LFS-tracked dataset, the time-zone-pinned Spark session and configuration in one file.

11. Challenges Faced

Challenge	What happened	How we resolved it
Negative R^2 in the first model	Predicting absolute price from absolute OHLC gave $R^2 ≈ -5$ on the held-out (later) period.	Reframed the target as <code>next_close - close</code> and used differenced / return features (PR #3).
Training/serving skew	The single-row predict path hard-coded <code>diff_ma_5 = diff_ma_10 = 0</code> because it received only two bars — two of seven features meant something different at serve time, silently.	Added <code>src/serving/features.py</code> mirroring <code>preprocess.py</code> , a 10-bar minimum history, and pytest tests that assert parity (ADR-010).
Time-zone dependent split and parquet	The CSV timestamps are naive IST; a UTC container and an IST laptop produced different instants and a split that moved by 5h30m.	Pinned <code>spark.sql.session.timeZone=Asia/Kolkata</code> and computed the cutoff in epoch seconds (ADR-205).
Overnight gaps in window features	Rolling means crossed session boundaries, mixing yesterday's close with today's open.	Partitioned all windows by <code>session_date</code> and dropped the first 9 bars of every session (ADR-012).
Parallel tracks colliding	Three tracks (Airflow, serving, model fixes) edited <code>prediction_server.py</code> ; one merge had to be resolved with -Xours.	Agreed a shared data contract (<code>schema.py</code>), feature-branch + PR workflow, and CI on every PR.
Docker / CI friction	Python 3.14 + Spark 4.2 needed Java 21; data/ was excluded by <code>.dockerignore</code> so training failed in compose; mypy complained about untyped yaml in CI; pytest could not import <code>src</code> .	Installed OpenJDK 21 in the image, fixed <code>.dockerignore</code> , added <code>types-PyYAML</code> and <code>pydantic</code> to the mypy hook, moved the test to the repo root, added a compose CI job.
Large dataset in Git	A 57 MB CSV bloats a normal repository.	Tracked it with Git LFS (Section 6); stopped tracking trained models in LFS and moved them to the MLflow registry.

12. Future Improvements

- **Persist MLflow state across container restarts** by pointing `MLFLOW_TRACKING_URI` at the mounted `mlflow_data` volume (`sqlite:///app/mlflow_data/mlflow.db`) so training runs and prediction logs survive docker compose down, and expose the MLflow UI as a compose service.
- **Serve from the registry in compose** — add the Airflow evaluate/register stages to the compose pipeline so the API always loads models:`/nifty-return-predictor@production` rather than a filesystem path.
- **Drift detection and alerting** — compare the rolling distribution of predicted changes (already tracked by `/metrics`) with the training distribution, export Prometheus metrics and dashboard them in Grafana, and trigger the Airflow DAG for retraining when drift is detected (the feedback arrow in the architecture diagram).
- **Streaming ingestion with Kafka** — replace the static CSV with live minute bars, which would make the Kafka option of the rubric meaningful and allow scheduled daily retraining.
- **Richer models and tuning** — enable the GBT regressor already stubbed in `config.yaml`, add cross-validated hyper-parameter search on a rolling-origin basis, and try longer horizons with volatility features.
- **Continuous delivery** — push the built image to GitHub Container Registry on merges to main and deploy with a tag, and add a smoke test that calls `/predict` against the started container in CI.
- **Data versioning at scale** — if the dataset starts changing or exceeds LFS quotas, migrate to DVC with an object-store remote while keeping the same pipeline stages.
- **Restore `/health` and `/metrics` on main** — the performance-monitoring endpoints merged in PR #8 were overwritten by a later -Xours merge and should be re-applied to the current `prediction_server.py`.

Appendix A. Repository Structure and Dependencies

```

├── conf/config.yaml          # paths, split, features, hyper-params, MLflow, Spark
├── dags/stock_pipeline_dag.py # Airflow DAG: ingest > validate > preprocess > train
                             # > evaluate > register
├── src/
│   ├── config.py, schema.py, validate.py # config loader, data contract, validation
│   ├── data/ingest.py, preprocess.py     # stages 1 and 3
│   ├── models/train.py, evaluate.py, register.py # stages 4-6
│   └── serving/features.py               # serving-side features (ADR-010)
├── models/train_pyspark_models.py # RF / LR training run by docker compose (MLflow)
├── prediction_server.py           # FastAPI service
├── test_serving_features.py       # pytest: training/serving feature parity
├── data/raw/NIFTY 100_minute.csv  # Git LFS
├── docs/architecture.svg
├── Dockerfile, docker-compose.yml, .devcontainer/
├── .github/workflows/pre-commit-action.yaml, docker-compose-action.yaml
└── .pre-commit-config.yaml, requirements.txt, README.md

```

Pinned dependencies: pyspark 4.2.0, mlflow 3.15.1, pandas 2.3.3, numpy 2.5.0, pyarrow 24.0.0, scikit-learn 1.9.0, PyYAML 6.0.3, fastapi[standard] 0.136.3, apache-airflow 3.3.0, apache-airflow-providers-standard 1.15.0, pytest; runtime Python 3.14 + OpenJDK 21 (Docker), Python 3.13 in CI.

A.1 Quick start

```

git clone https://github.com/sohang3112/stock-prediction-mlops
cd stock-prediction-mlops
git lfs install && git lfs pull          # fetch the dataset
pip install pre-commit && pre-commit install # local lint/type/test hooks
docker compose up --build                # train both models, then serve on :8000
open http://localhost:8000/docs          # interactive API docs

```